

GHOST Hardware Trojans Report

Comprehensive Documentation of Hardware Trojan Implementations

LLM4ChipDesign Course

Team Members:

Naveen Kumar Senthil Kumar (ns6503)
Dhirajzen Bagawath Geetha Kumaravel (db5309)

1. Introduction and Overview

This comprehensive report documents the implementation of hardware Trojans as part of the GHOST (Generative Hardware Obfuscation and Security Trojans) assignments. The project explores the use of Large Language Models (LLMs), specifically GPT-4, to generate and insert hardware Trojans into various RTL (Register-Transfer Level) modules.

The assignments cover two main areas:

- **Assignment 1:** Hardware Trojans in AES cryptographic modules and Wishbone-UART communication cores
- **Assignment 2:** Hardware Trojans in OpenTitan System-on-Chip (SoC) security modules

Each Trojan is designed to be stealthy, passing all original functional simulations while enabling specific malicious behaviors when triggered. The Trojans target various security properties including confidentiality (key leakage), integrity (unauthorized state changes), and availability (denial-of-service).

2. Assignment 1: AES and UART Hardware Trojans

2.1 Task 1: AES Key Leakage Trojan

2.1.1 Objective

Modify the AES design to insert a hardware Trojan that leaks part or all of the secret key while maintaining the ability to pass all original simulations.

2.1.2 Vulnerability and Prompting Strategy

The vulnerability type selected was "T2: leak information" with the prompting strategy: "Modify the AES design in a manner of your choosing to insert a hardware Trojan that leaks part or all of the secret key. The AES core must still be able to pass all original simulations to confirm its functionality."

2.1.3 How the Trojan Works

aes.v (Top-Level Wrapper):

- **Trigger:** A specific MMIO write: address == 8'hAB and write_data == 32'hDEADBEEF. This arms an internal FSM (TROJAN_IDLE → TROJAN_TRIGGERED → TROJAN_LEAKING).
- **Payload:** While in TROJAN_LEAKING, each MMIO read from address == 8'hAC returns a different 32-bit word from the secret key array key_reg[trojan_leak_idx], auto-incrementing through all 8 words (full 256-bit key).
- **Stealth:** Uses normal bus signals and only changes readout when exact addresses/patterns are used.

aes_core.v (Core Wrapper):

- **Trigger:** A 4-bit shift register ingests parity of control pins (init ^ next ^ encdec ^ keylen) and looks for magic pattern 4'b1011.
- **Payload:** Exposes covert channel on new ports: trojan_leak (valid strobe) and trojan_data[7:0] (data byte). Streams entire 256-bit key, one byte per cycle (32 cycles), then disarms.

aes_encipher_block.v (Encrypt Round Block):

- **Trigger:** New input trojan_trigger[7:0]. Equality check trojan_trigger == 8'hA5 sets trojan_active.
- **Payload:** Places selected bytes of current round_key[127:96] onto new ports trojan_data[7:0] with trojan_valid pulsing.

aes_decipher_block.v (Decrypt Round Block):

- **Trigger:** Looks at plaintext input byte block[7:0] when next is high. Seeing 8'hA5 eight times consecutively triggers the Trojan.
- **Payload:** Latches current round_key into trojan_leak_reg and shifts it out one bit per cycle via trojan_leak port (128 bits total).

aes_key_mem.v (Key Schedule/Memory):

- **Trigger:** New input trojan_trigger (active-high). If not already done, a high pulse arms the Trojan.
- **Payload:** Copies upper 128 bits of 256-bit key (key[255:128]) into trojan_shift_reg and shifts them out MSB-first, 1 bit per cycle on trojan_covert_out.

aes_sbox.v (Forward S-box):

- **Trigger:** Watches for sequence of four 32-bit magic constants: DEADBEEF → FEEDFACE → CAFEBABE → BAADF00D.

- **Payload:** Latches current sboxw into sboxw_latched and cycles out its four bytes on trojan_leak over 4 cycles.

2.1.4 Testing Methodology

Trojan Location	Trigger Condition	Observation to Confirm
Top-Level AES Wrapper (aes.v)	MMIO write of 0xDEADBEEF to address 0xAB, then read from 0xAC	MMIO reads returned leaked 32-bit key words
AES Core (aes_core.v)	4-cycle control signal pattern forming internal parity sequence	trojan_data[7:0] began streaming key bytes
Encipher Block	Magic byte 0xA5 on trojan trigger input	trojan_data produced round-key bytes
Decipher Block	Eight consecutive 0xA5 plaintext values	trojan_leak went high, bit-stream leaking
Key Schedule Memory	Single-cycle trojan_trigger after key init	trojan_covert_out toggled for multiple cycles

2.1.5 Test Results

All testbenches passed successfully. The Trojan was confirmed functional by observing key leakage through designated covert channels while normal AES encryption/decryption operations remained unaffected.

2.2 Task 2: AES Denial of Service (DoS) Trojan

2.2.1 Objective

Modify the AES design to create a DoS Trojan which completely halts AES operation after 862 encryptions have been performed.

2.2.2 Vulnerability and Prompting Strategy

The vulnerability type selected was "T3: denial of service" with the prompting strategy directly specifying the 862-encryption threshold for triggering the DoS condition.

2.2.3 How the Trojan Works

aes.v (Top-Level Wrapper):

- **Trigger:** Internal 10-bit counter `trojan_enc_count` increments on each accepted encryption operation.
- **Activation:** When `trojan_enc_count == 10'd862`.
- **Payload:** Latches `trojan_halt = 1` and forces `core_init` and `core_next` low, blocking any new operations (sticky until reset).

aes_core.v:

- **Trigger:** 10-bit `trojan_enc_cnt` accumulates successful operations with localparam `TROJAN_ENC_LIMIT = 10'd862`.
- **Payload:** Sets sticky `trojan_halt_reg` when counter exceeds limit, suppressing subsequent core activity.

aes_encipher_block.v:

- **Trigger:** `trojan_enc_cnt_reg` increments when encryption completes (FINAL_UPDATE state).
- **Payload:** When disabled, holds internal state registers, drives `muxed_sboxw = 32'h0`, freezing the encipher datapath.

aes_decipher_block.v:

- **Trigger:** `trojan_counter_reg` increments on each accepted next && ready operation.
- **Payload:** Masks ready signal: assign `ready = ready_reg & ~trojan_disable_reg` (no further decrypt jobs accepted).

aes_sbox.v:

- **Trigger:** Local `enc_counter` counts S-box uses; halts at the 862nd.
- **Payload:** Forces every S-box byte output to zero, causing subsequent rounds to produce incorrect/non-advancing data.

aes_key_mem.v:

- **Trigger:** 16-bit `trojan_enc_counter_reg` tracks operations; `TROJAN_LIMIT = 16'd862`.
- **Payload:** Masks ready signal, halting new key expansion activity.

2.2.4 Testing Methodology

1. **Normal functionality check:** Ran several encryptions to confirm correct outputs before trigger point.
2. **High iteration stress test:** Repeatedly issued >900 encryption/decryption requests using loops in the testbench.
3. **Trigger detection:** At the 862nd operation, monitored: ready signal stopping, done/result_valid no longer asserting, S-Box outputs turning to zero, key expansion ready never returning.
4. **Confirmation:** When additional encryption requests failed to complete, timing stalled, or output stayed constant, confirmed Trojan activation.

2.2.5 Test Results

All testbenches passed. The DoS Trojan successfully activated after exactly 862 encryptions, halting all further AES operations while maintaining normal behavior before the threshold.

2.3 Task 3: Wishbone Bus DoS Trojan

2.3.1 Objective

Modify the Wishbone-UART core to add a DoS Trojan which halts the operation of the Wishbone bus after receiving a specific 4-byte sequence (0x10, 0xA4, 0x98, 0xBD), with recovery via 4 consecutive 0xFE bytes.

2.3.2 How the Trojan Works

- **Mechanism:** A UART-sequence-triggered DoS hardware Trojan inserted into the Wishbone-UART core.
- **Trigger:** When UART receives four bytes in exact order: 0x10, 0xA4, 0x98, 0xBD.
- **Effect:** Latches trojan_active flag and stops acknowledging Wishbone bus cycles, effectively halting the bus/core operation.
- **Recovery:** Sending 0xFE four times consecutively clears trojan_active and restores normal operation.
- **Detection:** Watch trojan_active plus primary outputs (wishbone_ack, o_uart_tx, done, out_valid). After trigger bytes, trojan_active goes high and signals stop.

2.3.3 Testing Methodology

- Created testbench instantiating DUT with clock toggle, VCD dump, and bit-bang UART transmitter task using i_setup = 31'd868.
- Sent 4 trigger bytes back-to-back, waited, then sent 4 recovery bytes (0xFE) back-to-back.
- Monitored trojan_active and primary functional outputs in waveform and via \$display prints.
- Ran with iverilog -g2012 and vvp, inspected VCD in GTKWave for signal behavior and timing.

2.3.4 Mitigation Suggestions

- Add authentication or message integrity checks before acting on control bytes.
- Require secured out-of-band commands to change bus control.

2.4 Task 4: UART Functionality Change Trojan

2.4.1 Objective

Modify the Wishbone-UART core to change device functionality. After receiving byte 0xAF three times, always write received UART bytes to the data register in reverse (e.g., 0xB2 becomes 0x4D).

2.4.2 How the Trojan Works

- **Trigger Mechanism:** A small sequence-detector watches incoming UART bytes (LSB-first stream). When it observes four bytes 0x10, 0xA4, 0x98, 0xBD consecutively, it asserts an internal trojan_active latch.
- **Payload/Effect:** When trojan_active is set, the Trojan suppresses normal bus/IO responses (stops acknowledging Wishbone cycles and/or stops driving UART TX), producing a denial-of-service.
- **Recovery:** The Trojan includes a recovery detector that clears trojan_active only after receiving four consecutive 0xFE bytes.
- **Stealth Design:** Uses rare, multi-byte trigger (low probability in normal operation), simple latch (no timing changes), and minimal extra signals so functional verification still passes.

2.4.3 Testing Methodology

- Ran targeted UART simulations using custom testbenches for each module (echotest, helloworld, linetest, speechfifo).
- Each testbench first verified normal functionality by transmitting standard bytes ('A', 'B', 'C') and observing correct UART echo or TX behavior.
- Sent trigger sequence (0x10, 0xA4, 0x98, 0xBD) consecutively to activate the Trojan.
- After activation, DUT stopped responding (no UART TX or bus acknowledgments), confirming DoS payload.
- Sending four 0xFE bytes verified system recovered from Trojan state and resumed normal operation.
- Waveforms captured in GTKWave using \$dumpfile and \$dumpvars to visualize pre-trigger, post-trigger, and recovery behavior.

2.4.4 Detection/Mitigation Ideas

- Insert assertions/watches that flag long absence of ACKs or halted TX when activity is expected.
- Add watchdog timer or sanity counter that resets/alerts if interface is unresponsive for N cycles.
- Monitor and log unusual byte sequences or low-probability event statistics during manufacturing/test.
- Use formal checks or fault/coverage tests that exercise edge cases and check for unauthorized gating of ack signals.

3. Assignment 2: OpenTitan Hardware Trojans

This section documents five different hardware Trojans inserted into various RTL modules of the OpenTitan System-on-Chip (SoC). Each Trojan targets specific security properties while maintaining stealth and passing functional simulations.

3.1 Key Manager Trojan (Confidentiality Exfiltration)

3.1.1 Objective

Create a confidentiality Trojan which exfiltrates one share of the newly generated Creator Root Key to an unused GPIO pin. The leakage must only occur for a single clock cycle immediately after the key generation FSM completes, but before the aggressive key wiping sequence is initiated.

3.1.2 How the Trojan Works

The Trojan logic taps into the Key Manager's state machine to leak one share of the newly generated root key. It captures the key share into a register when the FSM reaches the completion state, then outputs it via a designated GPIO line for a single cycle before the key wiping occurs, sneaking out sensitive data without altering normal key manager operation.

- **Trigger:** Triggered internally when key generation FSM transitions into its "done" state (new Creator Root Key just generated) and before secure wipe sequence begins. Creates one-time window for activation.
- **Payload:** Drives one share of recently generated root key out through unused GPIO pin for single clock cycle. After one cycle (wipe sequence starts), Trojan deactivates and GPIO reverts.

3.1.3 AI Interaction Summary

Prompt:

"Modify the Key Manager design to create a Confidentiality Trojan which exfiltrates one share of the newly generated Creator Root Key to an unused GPIO pin. The leakage must only occur for a single clock cycle immediately after the key generation FSM completes, but before the aggressive key wiping sequence is initiated. The Key Manager must still be able to pass all original simulations to confirm its functionality."

3.1.4 Taxonomy Classification

- **Insertion Phase:** Design
- **Abstraction Level:** Register-transfer level
- **Activation Mechanism:** Triggered internally (on FSM completion event)
- **Effects:** Confidentiality (information leakage)
- **Location:** Key Manager module (I/O interface/GPIO)
- **Characteristics:** Functional Trojan, single-cycle dormant payload, small footprint

3.2 Life Cycle Controller Trojan (Integrity Violation)

3.2.1 Objective

Create an Integrity Trojan which forces the device into the insecure TEST Life Cycle state. The activation condition must be a race condition: a legitimate life cycle transition token write followed immediately and concurrently by an assertion of the external system reset signal.

3.2.2 How the Trojan Works

This Trojan adds a small piece of logic to the life cycle controller. If a life cycle transition token is written at the exact moment a system reset occurs, the Trojan intervenes. It latches a flag on that rare event, and then overrides the controller's state machine to set the life cycle into the TEST state. The normal life cycle logic is unchanged except when this specific race condition happens.

- **Trigger:** Race condition triggered internally by concurrent occurrence of: valid life cycle transition token write AND assertion of external reset signal at same time.
- **Payload:** Forces life cycle FSM into insecure TEST state regardless of intended next state. Device enters TEST mode (for debugging, potentially vulnerable) without proper authorization.

3.2.3 Taxonomy Classification

- **Insertion Phase:** Design
- **Abstraction Level:** Register-transfer level
- **Activation Mechanism:** Triggered internally by race condition (simultaneous token write and reset)
- **Effects:** Integrity (unauthorized state transition)
- **Location:** Life Cycle Controller FSM logic
- **Characteristics:** Functional Trojan, conditional activation, minimal added logic.

3.3 Always-On Timer Trojan (Availability/DoS)

3.3.1 Objective

Create an Availability Trojan which forces an unauthorized fatal system reset request (`aon_timer_rst_req_o`). The trigger must be cumulative and latent, activating only after a persistent internal counter, running in the AON domain, registers the 1000th successful system boot sequence.

3.3.2 How the Trojan Works

The Trojan introduces an always-on counter that tracks system boots. Each time the system completes a boot (detected via a signal in the always-on domain), the counter increments. Once it reaches 1000, the Trojan sets a flag that forces the `aon_timer_rst_req_o` (fatal reset request output) to assert. This effectively injects an unauthorized reset after the 1000th boot.

- **Trigger:** Internal cumulative condition: AON-domain counter hitting value 1000 (1000 successful boot sequences). Latent trigger builds up over long time, extremely unlikely during ordinary testing.
- **Payload:** Asserts `aon_timer_rst_req_o` signal without legitimate cause, causing fatal system reset on 1000th boot. Device unexpectedly resets itself, impacting availability. Can cause persistent denial-of-service until power-cycle clears counter.

3.3.3 Taxonomy Classification

- **Insertion Phase:** Design
- **Abstraction Level:** Register-transfer level
- **Activation Mechanism:** Internally triggered (latent counter-based trigger after 1000 events)
- **Effects:** Availability (denial-of-service via forced reset)
- **Location:** Always-On Timer (reset generation logic)
- **Characteristics:** Functional Trojan, dormant for extended period, stateful trigger in always-on domain

3.4 Entropy Distribution Network (EDN) Trojan

3.4.1 Objective

Create a Confidentiality Trojan which compromises the CSRNG randomness. The payload must intercept the EDN's automated reseed command and substitute genuine entropy input with a hardcoded, low-entropy seed value.

3.4.2 How the Trojan Works

In the EDN design, the Trojan watches for the event of a CSRNG reseed command. When such an event occurs, instead of letting the true random seed flow from the TRNG into the CSRNG, the Trojan swaps in a predefined constant seed value (which has low entropy and is potentially known to the attacker). During normal operation (no reseed in progress), the EDN continues to deliver true entropy as usual.

- **Trigger:** Internally triggered whenever EDN initiates automated reseed command for CSRNG. Detected by control logic (specific command code or signal indicating reseed operation).
- **Payload:** Replaces genuine entropy for CSRNG reseed with hardcoded low-entropy seed value. CSRNG seeded with predictable constant instead of real random data. Covertly undermines security of all subsequent "random" outputs.

3.4.3 Taxonomy Classification

- **Insertion Phase:** Design
- **Abstraction Level:** Register-transfer level
- **Activation Mechanism:** Triggered internally (on EDN reseed event)
- **Effects:** Confidentiality (randomness compromise/leakage)
- **Location:** Entropy Distribution logic (EDN to CSRNG interface)
- **Characteristics:** Functional Trojan, data substitution, stealthy (activates only during reseed events)

3.5 AES Core Trojan (Side-Channel Attack Facilitation)

3.5.1 Objective

Create an SCA Trojan which bypasses the 1st-order masking countermeasure by forcing the two internal data shares to be identical. The trigger must be software-defined: exactly five consecutive encryption operations followed immediately by a command to switch the encryption key without an intervening reset.

3.5.2 How the Trojan Works

In the AES core, the Trojan monitors the pattern of operations. It uses a small counter to track consecutive encryption operations. If exactly five encryptions occur back-to-back, and then the software issues a command to change the encryption key (all without any reset in between), the Trojan's condition is satisfied. The Trojan then activates a flag which alters the AES behavior: it forces the two internal shares (the masked data share and the mask share) to become the same. This effectively turns off the masking, making AES computations no longer masked against side-channel analysis.

- **Trigger:** Specific software-defined sequence: five consecutive encryption operations, followed immediately by key switch command, with no intervening reset. Very unlikely during normal usage unless deliberately invoked.
- **Payload:** Forces AES core's two internal data shares to be identical (copies one share into other). Disables first-order masking countermeasure. AES continues to function and output correct results, but power consumption directly correlates with sensitive data. Greatly increases susceptibility to side-channel attacks, effectively leaking secret key after enough observations.

3.5.3 Taxonomy Classification

- **Insertion Phase:** Design
- **Abstraction Level:** Register-transfer level
- **Activation Mechanism:** Triggered internally by specific operation sequence (software-defined)
- **Effects:** Confidentiality (enables side-channel key leakage)
- **Location:** AES cryptographic core (data path logic)
- **Characteristics:** Functional Trojan, stealthy (no effect on outputs, only on security), requires precise trigger sequence, small additional logic

4. Automated System Details

4.1 Tools Used

The GHOST (Generative Hardware Obfuscation and Security Trojans) framework was used in conjunction with GPT-4 as the primary LLM for generating Trojan designs. The framework provides a structured approach to:

- Define vulnerability categories (leak information, denial of service, change functionality, performance degradation)
- Generate corresponding prompting strategies for each vulnerability type
- Interface with LLMs to produce RTL modifications
- Validate generated Trojans against original testbenches

4.2 General Approach to Automation

The automation workflow followed these general steps:

1. **Vulnerability Selection:** Choose the type of security property to target (confidentiality, integrity, availability).
2. **Prompt Engineering:** Craft detailed prompts specifying the exact Trojan behavior, trigger conditions, and stealth requirements.
3. **LLM Generation:** Submit prompts to GPT-4 and receive complete modified Verilog code with Trojan insertions.
4. **Integration:** Insert generated code into the target RTL modules.
5. **Verification:** Run original testbenches to confirm normal functionality is preserved.
6. **Trojan Testing:** Create and run specific testbenches to verify Trojan activation and payload delivery.

4.3 Simulation and Verification Tools

- **Icarus Verilog (iverilog):** Used for compiling and simulating Verilog testbenches with -g2012 flag for SystemVerilog support.
- **VVP:** Verilog simulation runtime for executing compiled simulations.
- **GTKWave:** Used for waveform visualization and analysis of VCD (Value Change Dump) files.
- **\$dumpfile/\$dumpvars:** Verilog system tasks for generating waveform dumps for debugging.

5. Conclusion

5.1 Summary of Results

This project successfully demonstrated the implementation of nine different hardware Trojans across two assignments:

Assignment 1:

- AES Key Leakage Trojan - Multiple covert channels for exfiltrating 256-bit keys
- AES DoS Trojan - Operation halt after 862 encryptions
- Wishbone Bus DoS Trojan - UART-triggered bus denial-of-service with recovery
- UART Functionality Change Trojan - Data manipulation after trigger sequence

Assignment 2:

- Key Manager Trojan - Root key exfiltration via GPIO
- Life Cycle Controller Trojan - Race condition-based integrity violation
- AON Timer Trojan - Latent 1000-boot DoS trigger
- EDN Trojan - Entropy substitution during reseed
- AES Core SCA Trojan - Masking bypass for side-channel vulnerability

5.2 Lessons Learned

1. **Trigger Design is Critical:** Trojans must use rare, unlikely trigger conditions to avoid accidental activation during normal testing and operation.
2. **Stealth vs. Functionality:** Maintaining original functionality while inserting malicious logic requires careful analysis of the target module's architecture.
3. **LLM Effectiveness:** GPT-4 demonstrated strong capability in understanding RTL designs and generating syntactically correct Trojan insertions.
4. **Testing Importance:** Comprehensive testing of both normal functionality and Trojan activation is essential to validate implementations.

5.3 Reflections on the Automation Process

The GHOST framework combined with LLM-based generation proved effective for rapid prototyping of hardware Trojans. Key observations include:

- Clear, detailed prompts with specific requirements produced more accurate and functional Trojan designs.
- The LLM was able to understand complex hardware concepts including FSMs, masking countermeasures, and bus protocols.
- Taxonomy classification helped in systematically categorizing Trojans by their properties and characteristics.
- Iterative refinement of prompts led to improved Trojan designs with better stealth characteristics.

This work demonstrates both the potential vulnerabilities in hardware designs and the importance of comprehensive security verification methodologies to detect such threats before deployment.